XAI-Driven Analysis of Gesture Intent Detection using SHAP

This notebook is organized into runnable sections for SHAP stability, robustness under noise, negative motion simulation, and Gini-based intent gating.

Assumptions:

- `X_landmarks.npy` has shape `(N, 21, 3)`
- `y_labels.npy` has shape `(N,)`
- `gesture_model_best.pt` is in the working directory

If your checkpoint only stores `state_dict`, replace the `build_model()` placeholder with the architecture used during training.

```
# Environment setup and reproducibility
%pip install -q shap scikit-learn seaborn

import math
import random
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import shap
import torch
import torch.nn as nn
from IPython.display import display
from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_auc_score, roc_curve
from sklearn.model_selection import train_test_split

# Global configuration
DATA_CANDIDATES = [Path('.'), Path('/content'), Path('/content/drive/MyDrive')]

def resolve_data_path(filename: str) -> Path:
    """Find an uploaded file in the current notebook directory or common Colab locations."""
    for base_dir in DATA_CANDIDATES:
        candidate = base_dir / filename
        if candidate.exists():
            return candidate

    # Fallback: create placeholder assets in /content (Colab) or current directory.
    fallback_dir = Path('/content') if Path('/content').exists() else Path('.')
    try:
        if filename in {'X_landmarks.npy', 'y_labels.npy', 'gesture_model_best.pt'} and 'create_placeholder_assets' in globa
            x_file, y_file, model_file = create_placeholder_assets(fallback_dir)
            mapping = {
                'X_landmarks.npy': x_file,
                'y_labels.npy': y_file,
                'gesture_model_best.pt': model_file,
            }
            return mapping[filename]
    except Exception:
        pass

    searched = ', '.join(str(base_dir / filename) for base_dir in DATA_CANDIDATES)
    raise FileNotFoundError(f'Could not find {filename}. Searched: {searched}')

X_PATH = None
Y_PATH = None
MODEL_PATH = None

NUM_JOINTS = 21
NUM_COORDS = 3
INPUT_DIM = NUM_JOINTS * NUM_COORDS

# Experiment defaults. Increase these if you want a heavier but more rigorous run.
SEED = 42
TEST_SIZE = 0.2
N_BACKGROUND = 100
N_SHAP_EVAL = 200
SHAP_NSAMPLES = 128
SHAP_RUN_SEEDS = [0, 1, 2]
ROBUSTNESS_SIGMAS = np.linspace(0.0, 1.0, 10)
ROBUSTNESS_N_TRIALS = 10
ROBUSTNESS_MAX_SAMPLES = 50
NEGATIVE_SAMPLES_PER_TYPE = 50
NEGATIVE_SHAP_SAMPLES = 30
```

```
NEGATIVE_SHAP_SAMPLES = 50

sns.set_theme(style='whitegrid', context='notebook')
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.labelsize'] = 12


def set_seed(seed: int) -> None:
    """Set Python, NumPy, and PyTorch seeds for reproducibility."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False


set_seed(SEED)
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {DEVICE}')


def create_placeholder_assets(target_dir: Path = Path('.'), n_samples: int = 240, n_classes: int = 5):
    """Create synthetic landmark, label, and model files when real assets are unavailable."""
    target_dir = Path(target_dir)
    target_dir.mkdir(parents=True, exist_ok=True)

    x_file = target_dir / 'X_landmarks.npy'
    y_file = target_dir / 'y_labels.npy'
    model_file = target_dir / 'gesture_model_best.pt'

    if not x_file.exists() or not y_file.exists():
        rng = np.random.default_rng(SEED)
        X_placeholder = rng.normal(0.0, 1.0, size=(n_samples, NUM_JOINTS, NUM_COORDS)).astype(np.float32)
        y_placeholder = np.repeat(np.arange(n_classes, dtype=np.int64), n_samples // n_classes)
        if len(y_placeholder) < n_samples:
            extra = rng.integers(0, n_classes, size=n_samples - len(y_placeholder), dtype=np.int64)
            y_placeholder = np.concatenate([y_placeholder, extra])
        rng.shuffle(y_placeholder)
        np.save(x_file, X_placeholder)
        np.save(y_file, y_placeholder)

    if not model_file.exists():
        class _PlaceholderGestureModel(nn.Module):
            def __init__(self, num_classes: int):
                super().__init__()
                self.net = nn.Sequential(
                    nn.Flatten(),
                    nn.Linear(INPUT_DIM, 128),
                    nn.ReLU(),
                    nn.Linear(128, num_classes),
                )

            def forward(self, x):
                return self.net(x)

        model = _PlaceholderGestureModel(n_classes)
        with torch.no_grad():
            for param in model.parameters():
                param.uniform_(-0.05, 0.05)
        torch.save(model, model_file)

    return x_file, y_file, model_file


def load_dataset(x_path: Path | None = None, y_path: Path | None = None):
    """Load the landmark tensor and label vector from .npy files."""
    default_x, default_y, _ = create_placeholder_assets(Path('.'))
    x_path = default_x if x_path is None else Path(x_path)
    y_path = default_y if y_path is None else Path(y_path)
    if not x_path.exists() or not y_path.exists():
        fallback_x, fallback_y, _ = create_placeholder_assets(x_path.parent if x_path.parent.exists() else Path('.'))
        x_path = fallback_x if not x_path.exists() else x_path
        y_path = fallback_y if not y_path.exists() else y_path
    X = np.load(x_path).astype(np.float32)
    y = np.load(y_path).astype(np.int64)
    if X.ndim != 3 or X.shape[1:] != (NUM_JOINTS, NUM_COORDS):
        raise ValueError(f'Expected X shape (N, {NUM_JOINTS}, {NUM_COORDS}), got {X.shape}')
    if y.ndim != 1 or len(y) != len(X):
        raise ValueError(f'Expected y shape (N,), got {y.shape}')
    return X, y
```

```
def stratified_split(X: np.ndarray, y: np.ndarray, test_size: float = TEST_SIZE, seed: int = SEED):
    """Create a train/test split that preserves class balance."""
    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=test_size,
        random_state=seed,
        stratify=y,
    )
    return X_train, X_test, y_train, y_test


class GestureModelPlaceholder(nn.Module):
    """Replace this with the exact architecture used during training if the checkpoint stores only state_dict."""

    def __init__(self):
        super().__init__()
        raise NotImplementedError(
            'If gesture_model_best.pt is state_dict-only, define the training architecture inside build_model().'
        )


def build_model():
    """Return the exact trained architecture if the checkpoint needs to be restored from a state_dict."""
    return GestureModelPlaceholder()


def load_model(model_path: Path | None = None, device: torch.device = DEVICE):
    """Load a saved PyTorch model, TorchScript module, or a checkpoint dict.

    If the file contains only a state_dict, fill in build_model() with the architecture used in training.
    """
    _, _, default_model = create_placeholder_assets(Path('.'))
    model_path = default_model if model_path is None else Path(model_path)

    # TorchScript models should be loaded with torch.jit.load rather than torch.load.
    try:
        scripted_model = torch.jit.load(str(model_path), map_location=device)
        scripted_model.eval()
        return scripted_model
    except Exception:
        pass

    # PyTorch 2.6 may default to weights-only loading; request legacy behavior when available.
    try:
        checkpoint = torch.load(model_path, map_location=device, weights_only=False)
    except TypeError:
        checkpoint = torch.load(model_path, map_location=device)

    if isinstance(checkpoint, nn.Module):
        model = checkpoint.to(device)
        model.eval()
        return model

    if isinstance(checkpoint, dict):
        if 'model_state_dict' in checkpoint:
            model = build_model().to(device)
            model.load_state_dict(checkpoint['model_state_dict'])
            model.eval()
            return model
        if 'state_dict' in checkpoint:
            model = build_model().to(device)
            model.load_state_dict(checkpoint['state_dict'])
            model.eval()
            return model

    raise ValueError(
        'Unsupported checkpoint format. Save the full model, a TorchScript module, or provide build_model() for state_dict c
    )


def _forward_logits(model: nn.Module, x_tensor: torch.Tensor) -> torch.Tensor:
    """Call the model and normalize outputs to logits."""
    logits = model(x_tensor)
    if isinstance(logits, (tuple, list)):
        logits = logits[0]
    return logits


def predict_logits(model: nn.Module, X: np.ndarray, batch_size: int = 256, device: torch.device = DEVICE) -> np.ndarray:
    """Return logits for inputs of shape (N, 21, 3)."""
    model.eval()
```

```
    outputs = []
    with torch.no_grad():
        for start in range(0, len(X), batch_size):
            batch = torch.from_numpy(X[start:start + batch_size]).float().to(device)
            logits = _forward_logits(model, batch)
            outputs.append(logits.detach().cpu().numpy())
    return np.concatenate(outputs, axis=0)


def predict_proba(model: nn.Module, X: np.ndarray, batch_size: int = 256, device: torch.device = DEVICE) -> np.ndarray:
    """Return softmax probabilities for inputs of shape (N, 21, 3)."""
    logits = predict_logits(model, X, batch_size=batch_size, device=device)
    logits = logits - logits.max(axis=1, keepdims=True)
    exp_logits = np.exp(logits)
    return exp_logits / exp_logits.sum(axis=1, keepdims=True)


def predict_labels(model: nn.Module, X: np.ndarray, batch_size: int = 256, device: torch.device = DEVICE) -> np.ndarray:
    """Return predicted class indices."""
    return np.argmax(predict_proba(model, X, batch_size=batch_size, device=device), axis=1)


def predict_confidence(model: nn.Module, X: np.ndarray, batch_size: int = 256, device: torch.device = DEVICE) -> np.ndarray:
    """Return the maximum softmax probability for each sample."""
    probs = predict_proba(model, X, batch_size=batch_size, device=device)
    return probs.max(axis=1)


def compute_entropy(model: nn.Module, X: np.ndarray, batch_size: int = 256, device: torch.device = DEVICE) -> np.ndarray:
    """Return predictive entropy for each sample."""
    probs = predict_proba(model, X, batch_size=batch_size, device=device)
    eps = 1e-12
    return -np.sum(probs * np.log(probs + eps), axis=1)


def predict_proba_flat(model: nn.Module, X_flat: np.ndarray, batch_size: int = 256, device: torch.device = DEVICE) -> np.nda
    """Wrapper used by SHAP. Accepts flattened landmark arrays of shape (N, 63)."""
    X_flat = np.asarray(X_flat, dtype=np.float32)
    if X_flat.ndim == 1:
        X_flat = X_flat[None, :]
    X = X_flat.reshape(-1, NUM_JOINTS, NUM_COORDS)
    return predict_proba(model, X, batch_size=batch_size, device=device)


X, y = load_dataset()
X_train, X_test, y_train, y_test = stratified_split(X, y)

print('Dataset loaded successfully.')
print(f'Full dataset: X={X.shape}, y={y.shape}')
print(f'Train split:  X_train={X_train.shape}, y_train={y_train.shape}')
print(f'Test split:   X_test={X_test.shape}, y_test={y_test.shape}')
print('Class distribution:', dict(zip(*np.unique(y, return_counts=True))))
```

```
Using device: cpu
Dataset loaded successfully.
Full dataset: X=(2400, 21, 3), y=(2400,)
Train split:  X_train=(1920, 21, 3), y_train=(1920,)
Test split:   X_test=(480, 21, 3), y_test=(480,)
Class distribution: {np.int64(0): np.int64(405), np.int64(1): np.int64(399), np.int64(2): np.int64(418), np.int64(3): np.int
```

Upload Required Files

If the files are not already present in the runtime, upload them here before running the loading cells. In Colab, this will create the expected local files in `/content`.

```
# Upload the required files when running in Colab.
# In a local notebook or VS Code, this cell will just print a reminder.

def upload_required_files():
    try:
        from google.colab import files  # type: ignore
        uploaded = files.upload()
        print('Uploaded files:', list(uploaded.keys()))
    except Exception:
        print('Colab file upload is not available in this environment.')
        print('Place X_landmarks.npy, y_labels.npy, and gesture_model_best.pt in the notebook directory or /content.')


upload_required_files()
```

Train Model and Export Required Files (Colab)

Run the next cell to train a transformer-based gesture classifier and save:

- `X_landmarks.npy`
- `y_labels.npy`
- `gesture_model_best.pt`

If arrays are already uploaded, the cell trains on those. Otherwise, it creates a synthetic but learnable gesture dataset, trains the model, and exports all files to `/content` (or current directory outside Colab).

```
# Train a transformer classifier and export required files to Colab storage.

from pathlib import Path
from copy import deepcopy

import numpy as np
import torch
import torch.nn as nn
from sklearn.model_selection import train_test_split


def get_colab_output_dir() -> Path:
    return Path('/content') if Path('/content').exists() else Path('.')


def maybe_load_existing_arrays(output_dir: Path):
    x_path = output_dir / 'X_landmarks.npy'
    y_path = output_dir / 'y_labels.npy'
    if x_path.exists() and y_path.exists():
        X = np.load(x_path).astype(np.float32)
        y = np.load(y_path).astype(np.int64)
        if X.ndim == 3 and X.shape[1:] == (21, 3) and y.ndim == 1 and len(y) == len(X):
            print('Using existing uploaded arrays.')
            return X, y
    return None, None


def create_synthetic_dataset(n_samples: int = 2400, n_classes: int = 6, seed: int = 42):
    """Create a learnable landmark dataset for pipeline testing when real data is absent."""
    rng = np.random.default_rng(seed)
    class_prototypes = rng.normal(0.0, 1.0, size=(n_classes, 21, 3)).astype(np.float32)

    y = rng.integers(0, n_classes, size=n_samples, dtype=np.int64)
    X = np.empty((n_samples, 21, 3), dtype=np.float32)

    for i, label in enumerate(y):
        global_shift = rng.normal(0.0, 0.15, size=(1, 3)).astype(np.float32)
        per_joint_noise = rng.normal(0.0, 0.20, size=(21, 3)).astype(np.float32)
        X[i] = class_prototypes[label] + global_shift + per_joint_noise

    return X, y


class GestureTransformerClassifier(nn.Module):
    def __init__(self, num_classes: int, embed_dim: int = 64, num_heads: int = 4, num_layers: int = 2, dropout: float = 0.
        super().__init__()
        self.input_proj = nn.Linear(3, embed_dim)
        self.pos_embedding = nn.Parameter(torch.zeros(1, 21, embed_dim))

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim,
            nhead=num_heads,
            dim_feedforward=embed_dim * 4,
            dropout=dropout,
            batch_first=True,
            activation='gelu',
        )
        self.encoder = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
        self.norm = nn.LayerNorm(embed_dim)
        self.head = nn.Sequential(
            nn.Linear(embed_dim, embed_dim),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(embed_dim, num_classes),
        )

    def forward(self, x):
```

```
        x = self.input_proj(x)
        x = x + self.pos_embedding
        x = self.encoder(x)
        x = self.norm(x.mean(dim=1))
        return self.head(x)


def train_model(X: np.ndarray, y: np.ndarray, epochs: int = 20, batch_size: int = 128, lr: float = 1e-3, seed: int = 42):
    torch.manual_seed(seed)
    np.random.seed(seed)

    n_classes = int(np.max(y)) + 1
    X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=seed, stratify=y)

    train_ds = torch.utils.data.TensorDataset(
        torch.tensor(X_train, dtype=torch.float32),
        torch.tensor(y_train, dtype=torch.long),
    )
    val_ds = torch.utils.data.TensorDataset(
        torch.tensor(X_val, dtype=torch.float32),
        torch.tensor(y_val, dtype=torch.long),
    )

    train_loader = torch.utils.data.DataLoader(train_ds, batch_size=batch_size, shuffle=True)
    val_loader = torch.utils.data.DataLoader(val_ds, batch_size=batch_size, shuffle=False)

    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    model = GestureTransformerClassifier(num_classes=n_classes).to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-4)

    best_val_acc = 0.0
    best_state = deepcopy(model.state_dict())

    for epoch in range(1, epochs + 1):
        model.train()
        running_loss = 0.0
        running_correct = 0
        running_total = 0

        for xb, yb in train_loader:
            xb, yb = xb.to(device), yb.to(device)
            optimizer.zero_grad()
            logits = model(xb)
            loss = criterion(logits, yb)
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * xb.size(0)
            running_correct += (logits.argmax(dim=1) == yb).sum().item()
            running_total += xb.size(0)

        train_loss = running_loss / max(1, running_total)
        train_acc = running_correct / max(1, running_total)

        model.eval()
        val_correct = 0
        val_total = 0
        with torch.no_grad():
            for xb, yb in val_loader:
                xb, yb = xb.to(device), yb.to(device)
                logits = model(xb)
                val_correct += (logits.argmax(dim=1) == yb).sum().item()
                val_total += xb.size(0)
        val_acc = val_correct / max(1, val_total)

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            best_state = deepcopy(model.state_dict())

        print(f'Epoch {epoch:02d}/{epochs} | train_loss={train_loss:.4f} | train_acc={train_acc:.4f} | val_acc={val_acc:.4

    model.load_state_dict(best_state)
    model.eval()
    return model, best_val_acc


def export_required_files(output_dir: Path, X: np.ndarray, y: np.ndarray, model: nn.Module):
    output_dir.mkdir(parents=True, exist_ok=True)

    x_path = output_dir / 'X_landmarks.npy'
```

```
    y_path = output_dir / 'y_labels.npy'
    model_path = output_dir / 'gesture_model_best.pt'

    np.save(x_path, X.astype(np.float32))
    np.save(y_path, y.astype(np.int64))

    # Save as TorchScript for robust loading across sessions.
    scripted = torch.jit.script(model.cpu())
    scripted.save(str(model_path))

    print('\nExport complete:')
    print(f'- {x_path}')
    print(f'- {y_path}')
    print(f'- {model_path}')


RUN_TRAIN_AND_EXPORT = True

if RUN_TRAIN_AND_EXPORT:
    output_dir = get_colab_output_dir()

    X_existing, y_existing = maybe_load_existing_arrays(output_dir)
    if X_existing is None:
        X_data, y_data = create_synthetic_dataset(n_samples=2400, n_classes=6, seed=42)
        print('Created synthetic dataset because uploaded arrays were not found.')
    else:
        X_data, y_data = X_existing, y_existing

    trained_model, best_val_acc = train_model(X_data, y_data, epochs=20, batch_size=128, lr=1e-3, seed=42)
    print(f'Best validation accuracy: {best_val_acc:.4f}')

    export_required_files(output_dir, X_data, y_data, trained_model)

    # Keep these available for later notebook sections.
    X, y = X_data, y_data
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

```
Created synthetic dataset because uploaded arrays were not found.
Epoch 01/20 | train_loss=1.3225 | train_acc=0.5922 | val_acc=0.8167
Epoch 02/20 | train_loss=0.5591 | train_acc=0.9078 | val_acc=0.9646
Epoch 03/20 | train_loss=0.1729 | train_acc=0.9922 | val_acc=0.9979
Epoch 04/20 | train_loss=0.0477 | train_acc=1.0000 | val_acc=1.0000
Epoch 05/20 | train_loss=0.0200 | train_acc=1.0000 | val_acc=1.0000
Epoch 06/20 | train_loss=0.0115 | train_acc=1.0000 | val_acc=1.0000
Epoch 07/20 | train_loss=0.0082 | train_acc=1.0000 | val_acc=1.0000
Epoch 08/20 | train_loss=0.0063 | train_acc=1.0000 | val_acc=1.0000
Epoch 09/20 | train_loss=0.0051 | train_acc=1.0000 | val_acc=1.0000
Epoch 10/20 | train_loss=0.0041 | train_acc=1.0000 | val_acc=1.0000
Epoch 11/20 | train_loss=0.0034 | train_acc=1.0000 | val_acc=1.0000
Epoch 12/20 | train_loss=0.0029 | train_acc=1.0000 | val_acc=1.0000
Epoch 13/20 | train_loss=0.0026 | train_acc=1.0000 | val_acc=1.0000
Epoch 14/20 | train_loss=0.0022 | train_acc=1.0000 | val_acc=1.0000
Epoch 15/20 | train_loss=0.0020 | train_acc=1.0000 | val_acc=1.0000
Epoch 16/20 | train_loss=0.0017 | train_acc=1.0000 | val_acc=1.0000
Epoch 17/20 | train_loss=0.0016 | train_acc=1.0000 | val_acc=1.0000
Epoch 18/20 | train_loss=0.0014 | train_acc=1.0000 | val_acc=1.0000
Epoch 19/20 | train_loss=0.0013 | train_acc=1.0000 | val_acc=1.0000
Epoch 20/20 | train_loss=0.0012 | train_acc=1.0000 | val_acc=1.0000
Best validation accuracy: 1.0000

Export complete:
- /content/X_landmarks.npy
- /content/y_labels.npy
- /content/gesture_model_best.pt
```

SHAP Stability Analysis

This section computes KernelExplainer SHAP values on a fixed class-balanced background and repeats the analysis under multiple random seeds. The goal is to see whether the same joints stay important across runs.

```
# SHAP utilities and stability analysis

def select_class_balanced_indices(y_values: np.ndarray, n_total: int, seed: int = 0) -> np.ndarray:
    """Select roughly class-balanced indices with replacement only if needed."""
    rng = np.random.default_rng(seed)
    classes = np.unique(y_values)
    per_class = n_total // len(classes)
    remainder = n_total % len(classes)
    selected = []

    for class_index, class_label in enumerate(classes):
        class_indices = np.where(y_values == class_label)[0]
```

```
        n_take = per_class + (1 if class_index < remainder else 0)
        if len(class_indices) == 0:
            continue
        replace = len(class_indices) < n_take
        chosen = rng.choice(class_indices, size=n_take, replace=replace)
        selected.extend(chosen.tolist())

    selected = np.array(selected, dtype=int)
    if len(selected) < n_total:
        remaining = np.setdiff1d(np.arange(len(y_values)), selected, assume_unique=False)
        extra = rng.choice(remaining, size=n_total - len(selected), replace=False)
        selected = np.concatenate([selected, extra])

    rng.shuffle(selected)
    return selected[:n_total]


def select_eval_subset(X_values: np.ndarray, y_values: np.ndarray, n_samples: int, seed: int = 0):
    """Sample a deterministic evaluation subset from the test split."""
    rng = np.random.default_rng(seed)
    n_samples = min(n_samples, len(X_values))
    indices = rng.choice(len(X_values), size=n_samples, replace=False)
    return X_values[indices], y_values[indices], indices


def compute_shap(model: nn.Module, background: np.ndarray, eval_samples: np.ndarray, nsamples: int = SHAP_NSAMPLES):
    """Compute KernelExplainer SHAP values on flattened landmark inputs.

    Returns a list of SHAP arrays, one per class for multi-class models.
    """
    background_flat = background.reshape(len(background), -1)
    eval_flat = eval_samples.reshape(len(eval_samples), -1)

    explainer = shap.KernelExplainer(
        lambda x: predict_proba_flat(model, x, batch_size=256, device=DEVICE),
        background_flat,
    )
    shap_values = explainer.shap_values(eval_flat, nsamples=nsamples)
    return explainer, shap_values


def get_sample_shap_vector(shap_values, sample_index: int, class_index: int) -> np.ndarray:
    """Extract a single flattened SHAP vector for the requested sample and class."""
    if isinstance(shap_values, list):
        return np.asarray(shap_values[class_index][sample_index], dtype=np.float32)
    return np.asarray(shap_values[sample_index], dtype=np.float32)


def aggregate_joint_importance(flat_shap_values: np.ndarray) -> np.ndarray:
    """Aggregate flattened SHAP values into joint-level importance scores.

    Input can be shape (63,) or (N, 63). The return shape is (21,) or (N, 21).
    """
    flat_shap_values = np.asarray(flat_shap_values, dtype=np.float32)
    if flat_shap_values.ndim == 1:
        flat_shap_values = flat_shap_values[None, :]
    joint_scores = np.abs(flat_shap_values).reshape(-1, NUM_JOINTS, NUM_COORDS).sum(axis=2)
    return joint_scores.squeeze()


def top_k_overlap(run_joint_importances: np.ndarray, k: int = 5) -> float:
    """Average overlap of top-k joints across runs."""
    top_sets = [set(np.argsort(-row)[:k].tolist()) for row in run_joint_importances]
    overlaps = []
    for i in range(len(top_sets)):
        for j in range(i + 1, len(top_sets)):
            overlaps.append(len(top_sets[i] & top_sets[j]) / float(k))
    return float(np.mean(overlaps)) if overlaps else 0.0


def plot_joint_importance_with_errorbars(mean_importance: np.ndarray, std_importance: np.ndarray, title: str = 'Mean SHAP i
    joint_labels = [f'J{i + 1}' for i in range(NUM_JOINTS)]
    order = np.argsort(-mean_importance)
    plt.figure(figsize=(14, 6))
    plt.bar(np.array(joint_labels)[order], mean_importance[order], yerr=std_importance[order], capsize=4, color='#4C78A8')
    plt.ylabel('Aggregated SHAP importance')
    plt.xlabel('Joint')
    plt.title(title)
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

```
def run_shap_stability_analysis(
    model: nn.Module,
    X_train_values: np.ndarray,
    y_train_values: np.ndarray,
    X_test_values: np.ndarray,
    y_test_values: np.ndarray,
    background_size: int = N_BACKGROUND,
    eval_size: int = N_SHAP_EVAL,
    seeds = SHAP_RUN_SEEDS,
    nsamples: int = SHAP_NSAMPLES,
):
    """Repeat SHAP analysis under multiple random seeds and summarize joint stability."""
    all_run_joint_importances = []
    all_run_details = []

    for run_seed in seeds:
        set_seed(run_seed)
        background_idx = select_class_balanced_indices(y_train_values, background_size, seed=run_seed)
        eval_X, eval_y, eval_idx = select_eval_subset(X_test_values, y_test_values, eval_size, seed=run_seed)
        background = X_train_values[background_idx]

        _, shap_values = compute_shap(model, background, eval_X, nsamples=nsamples)
        preds = predict_labels(model, eval_X)

        sample_joint_importances = []
        for i in range(len(eval_X)):
            class_idx = int(preds[i])
            sample_flat_shap = get_sample_shap_vector(shap_values, i, class_idx)
            sample_joint_importances.append(aggregate_joint_importance(sample_flat_shap))

        sample_joint_importances = np.asarray(sample_joint_importances, dtype=np.float32)
        mean_joint_importance = sample_joint_importances.mean(axis=0)

        all_run_joint_importances.append(mean_joint_importance)
        all_run_details.append(
            {
                'seed': run_seed,
                'background_idx': background_idx,
                'eval_idx': eval_idx,
                'eval_y': eval_y,
                'sample_joint_importances': sample_joint_importances,
                'mean_joint_importance': mean_joint_importance,
            }
        )

        print(f'[SHAP] Seed {run_seed}: computed SHAP for {len(eval_X)} samples with background size {len(background)}')

    all_run_joint_importances = np.asarray(all_run_joint_importances, dtype=np.float32)
    mean_importance = all_run_joint_importances.mean(axis=0)
    std_importance = all_run_joint_importances.std(axis=0)

    print('\nSHAP stability summary')
    print('Top-5 joint overlap across runs:', round(top_k_overlap(all_run_joint_importances, k=5), 4))
    print('Top-5 joints by mean importance:', np.argsort(-mean_importance)[:5].tolist())

    return {
        'run_joint_importances': all_run_joint_importances,
        'mean_importance': mean_importance,
        'std_importance': std_importance,
        'run_details': all_run_details,
    }


# Example call for the SHAP stability experiment.
# Run this cell when you want the full analysis; it is intentionally not executed automatically.
RUN_SHAP_STABILITY = False
if RUN_SHAP_STABILITY:
    shap_stability_results = run_shap_stability_analysis(
        model=load_model(),
        X_train_values=X_train,
        y_train_values=y_train,
        X_test_values=X_test,
        y_test_values=y_test,
    )
    plot_joint_importance_with_errorbars(
        shap_stability_results['mean_importance'],
        shap_stability_results['std_importance'],
        title='SHAP stability across seeds: mean ± std per joint',
    )
```

Interpretation note

If the same few joints remain on top across seeds and the error bars stay small relative to the mean, the SHAP attribution pattern is stable. Large standard deviations or frequent ranking changes suggest the explanation is fragile to background selection or sampling noise.

Fragility, Robustness, and Negative Motion

This section first measures how often predictions flip under Gaussian noise, then generates non-intent inputs to compare confidence, SHAP concentration, and Gini score distributions against real gestures.

```
# Robustness analysis, negative sample generation, and Gini computation

def add_noise(X_values: np.ndarray, sigma: float, joint_index: int | None = None, seed: int = 0) -> np.ndarray:
    """Add Gaussian noise to all joints or to one selected joint only."""
    if sigma <= 0:
        return X_values.copy().astype(np.float32)

    rng = np.random.default_rng(seed)
    noisy = X_values.copy().astype(np.float32)

    if joint_index is None:
        noisy += rng.normal(0.0, sigma, size=noisy.shape).astype(np.float32)
    else:
        joint_noise = rng.normal(0.0, sigma, size=(len(noisy), NUM_COORDS)).astype(np.float32)
        noisy[:, joint_index, :] += joint_noise

    return noisy


def compute_gini(values: np.ndarray, eps: float = 1e-12) -> float:
    """Compute the Gini coefficient over non-negative values."""
    x = np.asarray(values, dtype=np.float64).reshape(-1)
    x = np.abs(x)
    if np.allclose(x.sum(), 0.0):
        return 0.0
    x = np.sort(x)
    n = len(x)
    index = np.arange(1, n + 1, dtype=np.float64)
    gini = (2.0 * np.sum(index * x) / (n * (x.sum() + eps))) - (n + 1.0) / n
    return float(np.clip(gini, 0.0, 1.0))


def run_robustness_analysis(
    model: nn.Module,
    X_eval: np.ndarray,
    sigmas = ROBUSTNESS_SIGMAS,
    n_trials: int = ROBUSTNESS_N_TRIALS,
    max_samples: int = ROBUSTNESS_MAX_SAMPLES,
    seed: int = SEED,
):
    """Measure how often predictions flip under noisy perturbations.

    Returns:
    - sample_flip_prob: shape (n_samples, n_sigmas)
    - joint_flip_prob: shape (n_sigmas, 21)
    """
    rng = np.random.default_rng(seed)
    n_eval = min(max_samples, len(X_eval))
    sample_indices = rng.choice(len(X_eval), size=n_eval, replace=False)
    X_subset = X_eval[sample_indices]
    base_preds = predict_labels(model, X_subset)

    sample_flip_prob = np.zeros((n_eval, len(sigmas)), dtype=np.float32)
    joint_flip_prob = np.zeros((len(sigmas), NUM_JOINTS), dtype=np.float32)

    for sigma_index, sigma in enumerate(sigmas):
        # Per-sample flip probability.
        for sample_index in range(n_eval):
            flips = 0
            x_clean = X_subset[sample_index:sample_index + 1]
            for trial in range(n_trials):
                noisy = add_noise(x_clean, sigma=sigma, seed=seed + 1000 * sigma_index + 10 * sample_index + trial)
                pred = predict_labels(model, noisy)[0]
                flips += int(pred != base_preds[sample_index])
            sample_flip_prob[sample_index, sigma_index] = flips / float(n_trials)

        # Per-joint flip probability, perturbing one joint at a time.
```

```
        for joint_index in range(NUM_JOINTS):
            flips = 0
            total = 0
            for sample_index in range(n_eval):
                x_clean = X_subset[sample_index:sample_index + 1]
                for trial in range(n_trials):
                    noisy = add_noise(
                        x_clean,
                        sigma=sigma,
                        joint_index=joint_index,
                        seed=seed + 5000 * sigma_index + 100 * joint_index + 10 * sample_index + trial,
                    )
                    pred = predict_labels(model, noisy)[0]
                    flips += int(pred != base_preds[sample_index])
                    total += 1
            joint_flip_prob[sigma_index, joint_index] = flips / float(total)

        print(f'[Robustness] sigma={sigma:.3f} completed')

    overall_curve = sample_flip_prob.mean(axis=0)
    overall_std = sample_flip_prob.std(axis=0)

    return {
        'sample_indices': sample_indices,
        'base_preds': base_preds,
        'sample_flip_prob': sample_flip_prob,
        'joint_flip_prob': joint_flip_prob,
        'overall_curve': overall_curve,
        'overall_std': overall_std,
        'sigmas': np.asarray(sigmas, dtype=np.float32),
    }


def plot_robustness_curve(sigmas: np.ndarray, overall_curve: np.ndarray, overall_std: np.ndarray):
    plt.figure(figsize=(10, 5))
    plt.errorbar(sigmas, overall_curve, yerr=overall_std, marker='o', linewidth=2, capsize=4, color='#F58518')
    plt.xlabel('Noise sigma')
    plt.ylabel('Prediction flip probability')
    plt.title('Gaussian noise robustness: flip probability vs sigma')
    plt.ylim(0.0, 1.0)
    plt.tight_layout()
    plt.show()


def plot_sample_flip_heatmap(sample_flip_prob: np.ndarray, sigmas: np.ndarray):
    plt.figure(figsize=(12, 6))
    sns.heatmap(sample_flip_prob, cmap='mako', vmin=0.0, vmax=1.0, xticklabels=[f'{s:.2f}' for s in sigmas])
    plt.xlabel('Sigma')
    plt.ylabel('Sample index')
    plt.title('Per-sample flip probability under Gaussian noise')
    plt.tight_layout()
    plt.show()


def plot_joint_fragility_heatmap(joint_flip_prob: np.ndarray, sigmas: np.ndarray):
    plt.figure(figsize=(14, 6))
    joint_labels = [f'J{i + 1}' for i in range(NUM_JOINTS)]
    sns.heatmap(
        joint_flip_prob,
        cmap='rocket_r',
        vmin=0.0,
        vmax=1.0,
        xticklabels=joint_labels,
        yticklabels=[f'{s:.2f}' for s in sigmas],
    )
    plt.xlabel('Joint')
    plt.ylabel('Sigma')
    plt.title('Per-joint fragility heatmap: flip probability when one joint is perturbed')
    plt.tight_layout()
    plt.show()


def generate_negative_samples(X_reference: np.ndarray, n_samples: int = NEGATIVE_SAMPLES_PER_TYPE, seed: int = SEED):
    """Create three classes of negative motion: Gaussian noise, joint shuffle, and temporal jitter.

    Temporal jitter is a proxy because the input has no explicit time axis.
    """
    rng = np.random.default_rng(seed)
    n_samples = min(n_samples, len(X_reference))
    base_indices = rng.choice(len(X_reference), size=n_samples, replace=False)
    base = X_reference[base_indices].astype(np.float32)
```

```
    data_scale = float(np.std(X_reference) + 1e-6)
    gaussian_noise = rng.normal(0.0, data_scale, size=base.shape).astype(np.float32)

    shuffled_joints = np.empty_like(base)
    for i, sample in enumerate(base):
        permutation = rng.permutation(NUM_JOINTS)
        shuffled_joints[i] = sample[permutation]

    jitter = base.copy()
    smooth_noise = rng.normal(0.0, 0.05 * data_scale, size=base.shape).astype(np.float32)
    smooth_noise = np.cumsum(smooth_noise, axis=1)
    jitter += smooth_noise

    return {
        'real': base,
        'gaussian_noise': gaussian_noise,
        'shuffled_joints': shuffled_joints,
        'temporal_jitter': jitter,
    }


def _shap_and_gini_for_batch(model: nn.Module, X_batch: np.ndarray, background: np.ndarray, nsamples: int, seed: int = 0):
    """Compute SHAP, joint importance, Gini, confidence, and entropy for a batch."""
    set_seed(seed)
    _, shap_values = compute_shap(model, background, X_batch, nsamples=nsamples)
    preds = predict_labels(model, X_batch)
    probs = predict_proba(model, X_batch)
    conf = probs.max(axis=1)
    entropy = -np.sum(probs * np.log(probs + 1e-12), axis=1)

    joint_importances = []
    ginis = []
    for i in range(len(X_batch)):
        class_idx = int(preds[i])
        sample_flat_shap = get_sample_shap_vector(shap_values, i, class_idx)
        sample_joint_importance = aggregate_joint_importance(sample_flat_shap)
        joint_importances.append(sample_joint_importance)
        ginis.append(compute_gini(sample_joint_importance))

    return {
        'preds': preds,
        'probs': probs,
        'confidence': conf,
        'entropy': entropy,
        'joint_importance': np.asarray(joint_importances, dtype=np.float32),
        'gini': np.asarray(ginis, dtype=np.float32),
    }


def run_negative_motion_analysis(
    model: nn.Module,
    X_train_values: np.ndarray,
    y_train_values: np.ndarray,
    X_real_eval: np.ndarray,
    n_negative: int = NEGATIVE_SAMPLES_PER_TYPE,
    n_shap: int = NEGATIVE_SHAP_SAMPLES,
    seed: int = SEED,
):
    """Compare real gestures and negative motion under confidence, SHAP, and Gini."""
    background_idx = select_class_balanced_indices(y_train_values, N_BACKGROUND, seed=seed)
    background = X_train_values[background_idx]
    negatives = generate_negative_samples(X_real_eval, n_samples=n_negative, seed=seed)
    real_subset = X_real_eval[:n_negative].astype(np.float32)

    results = {}
    for label, samples in [('real', real_subset), *[(k, v) for k, v in negatives.items() if k != 'real']]:
        sample_count = min(n_shap, len(samples))
        samples_for_shap = samples[:sample_count]
        batch_stats = _shap_and_gini_for_batch(model, samples_for_shap, background, nsamples=SHAP_NSAMPLES, seed=seed)
        results[label] = {
            'samples': samples,
            'confidence': predict_confidence(model, samples),
            'entropy': compute_entropy(model, samples),
            'shap_gini': batch_stats['gini'],
            'joint_importance': batch_stats['joint_importance'],
            'preds': batch_stats['preds'],
        }
        print(f'[Negative motion] Processed {label}: {len(samples)} samples, SHAP on {sample_count} samples')

    return results
```

```
def plot_real_vs_negative_distributions(negative_results: dict):
    gini_records = []
    confidence_records = []

    for label, stats in negative_results.items():
        for value in stats['shap_gini']:
            gini_records.append({'group': label, 'value': float(value)})
        for value in stats['confidence']:
            confidence_records.append({'group': label, 'value': float(value)})

    gini_df = pd.DataFrame(gini_records)
    confidence_df = pd.DataFrame(confidence_records)

    fig, axes = plt.subplots(1, 2, figsize=(16, 5))
    sns.boxplot(data=gini_df, x='group', y='value', ax=axes[0], palette='Set2')
    sns.stripplot(data=gini_df, x='group', y='value', ax=axes[0], color='black', size=3, alpha=0.35)
    axes[0].set_title('Gini of SHAP: real vs random motion')
    axes[0].set_xlabel('Input group')
    axes[0].set_ylabel('Gini coefficient')

    sns.boxplot(data=confidence_df, x='group', y='value', ax=axes[1], palette='Set2')
    sns.stripplot(data=confidence_df, x='group', y='value', ax=axes[1], color='black', size=3, alpha=0.35)
    axes[1].set_title('Softmax confidence: real vs random motion')
    axes[1].set_xlabel('Input group')
    axes[1].set_ylabel('Max softmax probability')

    plt.tight_layout()
    plt.show()


RUN_ROBUSTNESS = False
RUN_NEGATIVE_MOTION = False
if RUN_ROBUSTNESS:
    model = load_model()
    robustness_results = run_robustness_analysis(model, X_test)
    plot_robustness_curve(robustness_results['sigmas'], robustness_results['overall_curve'], robustness_results['overall_st
    plot_sample_flip_heatmap(robustness_results['sample_flip_prob'], robustness_results['sigmas'])
    plot_joint_fragility_heatmap(robustness_results['joint_flip_prob'], robustness_results['sigmas'])

if RUN_NEGATIVE_MOTION:
    model = load_model()
    negative_motion_results = run_negative_motion_analysis(model, X_train, y_train, X_test)
    plot_real_vs_negative_distributions(negative_motion_results)
```

Interpretation note

A steep rise in flip probability at small sigma means the classifier is fragile to noise. For negative motion, real gestures should usually show higher confidence and a different SHAP concentration pattern than random inputs; if Gini overlaps heavily, the intent gate will be weak.

Gini-Based Intent Gate Evaluation

This section treats higher score as stronger intent. Gini, maximum softmax probability, and negative entropy are all evaluated with the same threshold-sweep and ROC machinery so the comparison is direct.

```
# Intent gate evaluation with Gini, max-softmax, and entropy baselines

def evaluate_gate(y_true: np.ndarray, score: np.ndarray, higher_is_intent: bool = True, n_thresholds: int = 200):
    """Sweep thresholds and return accuracy, precision, recall, and AUROC."""
    y_true = np.asarray(y_true).astype(int)
    score = np.asarray(score).astype(float)

    if not higher_is_intent:
        score = -score

    thresholds = np.linspace(score.min(), score.max(), n_thresholds)
    records = []
    best_record = None

    for threshold in thresholds:
        y_pred = (score >= threshold).astype(int)
        acc = accuracy_score(y_true, y_pred)
        prec = precision_score(y_true, y_pred, zero_division=0)
        rec = recall_score(y_true, y_pred, zero_division=0)
        record = {
            'threshold': float(threshold),
            'accuracy': float(acc),
```

```
            'precision': float(prec),
            'recall': float(rec),
        }
        records.append(record)
        if best_record is None or acc > best_record['accuracy']:
            best_record = record

    auroc = roc_auc_score(y_true, score)
    fpr, tpr, roc_thresholds = roc_curve(y_true, score)
    best_record['auroc'] = float(auroc)
    best_record['fpr'] = fpr
    best_record['tpr'] = tpr
    best_record['roc_thresholds'] = roc_thresholds
    best_record['all_thresholds'] = records
    return best_record


def build_gate_inputs(negative_results: dict):
    """Align real and random-motion arrays so every method uses the same sample count."""
    real_count = len(negative_results['real']['shap_gini'])
    y_true = np.concatenate([np.ones(real_count, dtype=int), np.zeros(real_count * 3, dtype=int)])

    real_conf = negative_results['real']['confidence'][:real_count]
    real_entropy = negative_results['real']['entropy'][:real_count]
    real_gini = negative_results['real']['shap_gini']

    negative_conf = []
    negative_entropy = []
    negative_gini = []
    for key in ['gaussian_noise', 'shuffled_joints', 'temporal_jitter']:
        negative_conf.append(negative_results[key]['confidence'][:real_count])
        negative_entropy.append(negative_results[key]['entropy'][:real_count])
        negative_gini.append(negative_results[key]['shap_gini'][:real_count])

    scores = {
        'gini': np.concatenate([real_gini, *negative_gini]),
        'max_softmax': np.concatenate([real_conf, *negative_conf]),
        'neg_entropy': np.concatenate([-real_entropy, *[-values for values in negative_entropy]]),
    }
    return y_true, scores


def plot_gate_roc_curves(results_by_method: dict):
    plt.figure(figsize=(8, 7))
    for method_name, result in results_by_method.items():
        plt.plot(result['fpr'], result['tpr'], linewidth=2, label=f"{method_name} (AUC={result['auroc']:.3f})")
    plt.plot([0, 1], [0, 1], linestyle='--', color='gray', linewidth=1)
    plt.xlabel('False positive rate')
    plt.ylabel('True positive rate')
    plt.title('ROC curves for intent gate methods')
    plt.legend()
    plt.tight_layout()
    plt.show()


def plot_gate_summary_table(results_by_method: dict):
    rows = []
    for method_name, result in results_by_method.items():
        rows.append(
            {
                'method': method_name,
                'best_threshold': result['threshold'],
                'accuracy': result['accuracy'],
                'precision': result['precision'],
                'recall': result['recall'],
                'AUROC': result['auroc'],
            }
        )
    table = pd.DataFrame(rows).sort_values('AUROC', ascending=False)
    display(table)
    return table


def run_intent_gate_evaluation(negative_results: dict):
    """Evaluate the gate using Gini, max-softmax, and negative entropy."""
    y_true, scores = build_gate_inputs(negative_results)
    results_by_method = {
        'Gini': evaluate_gate(y_true, scores['gini'], higher_is_intent=True),
        'Max softmax': evaluate_gate(y_true, scores['max_softmax'], higher_is_intent=True),
        'Entropy (negated)': evaluate_gate(y_true, scores['neg_entropy'], higher_is_intent=True),
    }
    plot_gate_roc_curves(results_by_method)
    table = plot_gate_summary_table(results_by_method)
```

```
    return results_by_method, table


RUN_GATE_EVALUATION = False
if RUN_GATE_EVALUATION:
    if 'negative_motion_results' not in globals():
        model = load_model()
        negative_motion_results = run_negative_motion_analysis(model, X_train, y_train, X_test)
    gate_results, gate_table = run_intent_gate_evaluation(negative_motion_results)
```

Run Order and Practical Notes

Suggested order:

1. Execute the setup and data/model cell.
2. Set `RUN_SHAP_STABILITY = True` and run the SHAP section.
3. Set `RUN_ROBUSTNESS = True` and run the robustness section.
4. Set `RUN_NEGATIVE_MOTION = True` and run the negative-motion section.
5. Set `RUN_GATE_EVALUATION = True` and run the gate comparison section.

For long Colab runs, start with smaller `N_SHAP_EVAL`, `SHAP_NSAMPLES`, or `NEGATIVE_SHAP_SAMPLES` if you want a quicker smoke test before running the full analysis.